Intermediate (build the loop yourself) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `react_loop_practice.py` (Intermediate section): an agent is not a new idea — it's Doc06's tool round trip, repeated until the model stops asking for tools. Building the loop yourself, once, is what makes `create_agent` and Project 2's Worker readable instead of magic. **If not:** the Build Task's `agent.py` would be the first loop you ever wrote, with no smaller version to trust.

Solution

Story — `run_agent()`: the loop becomes a function a caller can use: it returns an `AgentResult` (the answer plus every step), and running out of steps raises `MaxIterationsExceeded` with the partial log. **If not:** a run that never finished would look the same as one that finished with an empty answer.

```
# react_loop_practice.py
from dataclasses import dataclass, field

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


@tool
def add(a: int, b: int) -> int:
    """Add two integers and return the result."""
    return a + b


class MaxIterationsExceeded(Exception):
    def __init__(self, message: str, steps: list[dict]) -> None:
        # why: the caller needs the partial log to see why it never
        # finished — the same "error with details" idea as Doc02's
        # PermanentHTTPError
        self.steps = steps
        super().__init__(message)

@dataclass
class AgentResult:
    final_answer: str
```

```
    # why: a list default needs field(default_factory=list), not `= []` —
    # otherwise every AgentResult would share one list
    steps: list[dict] = field(default_factory=list)

def run_agent(task: str, max_iterations: int = 5) -> AgentResult:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([add])
    messages = [{"role": "user", "content": task}]
    steps: list[dict] = []

    for step in range(max_iterations):
        # THINK: the model answers now, or asks for a tool
        response = model_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            # when: no tool call means the model is done — the only exit
            return AgentResult(final_answer=response.content, steps=steps)

        for call in response.tool_calls:
            # ACT, then OBSERVE: run the tool, send the result back
            result = str(add.invoke(call["args"]))
            messages.append(
                {"role": "tool", "content": result, "tool_call_id": call["id"]}
            )
            steps.append({"step": step, "tool": call["name"], "result": result})

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)


# Run it on a task that needs two tool calls in a row.
result = run_agent("Add 1 + 2, then add 3 to that.")
print(f"Final answer: {result.final_answer}")
for step in result.steps:
    print(step)
```

Expected output (model wording varies):

```
Final answer: 1 + 2 is 3, and adding 3 gives 6.
{'step': 0, 'tool': 'add', 'result': '3'}
{'step': 1, 'tool': 'add', 'result': '6'}
```